

项目书完整版

面向昇腾 NPU 的技能驱动算子自动生成与硬件反馈优化系统

项目	内容
赛题方向	基于 AI/Agent 的 NPU 算子自动生成
团队名称	算子炼金术师
负责人	于天池（帕多瓦大学博士生） & 郑遣俊（慕尼黑工业大学博士生）
联系方式	yu-tianchi@outlook.com
版本	完整版 v1.0
日期	2026 年 7 月

摘要

SketchSkill-AGK 面向 CCF x MindSpore 昇腾 AGK 赛题，构建一个用于昇腾 910 NPU Benchmark 的技能驱动算子自动生成与硬件反馈优化系统。项目不把目标定义为一次性手写若干优化 kernel，而是构建可复用的自动化流程：从 AKG Bench Lite 任务中抽取 OpSpec，生成 NPU-aware Operator Sketch，检索算子模式 Skill Library，生成 Triton-Ascend 候选 ModelNew，在真实昇腾 NPU 上完成编译、运行、正确性验证、Pass@N 统计、性能测量、失败归因、修复和经验写回。

当前原型已经完成官方 Benchmark 固定版本管理、AKG Bench Lite 全部 13 个 case 的 OpSpec/Sketch 覆盖、候选提交布局、结果导入、Pass@N 汇总、Prompt 模板、Skill Library、确定性 replay 生成 provider、OpenAI Responses live provider 适配器、OpSpec/Sketch validator、package hygiene 测试，以及多个真实昇腾硬件实验。当前可用于初赛的总括性结论是：Lite benchmark OpSpec coverage: 13/13. Current executable candidates and Pass@4 evidence cover a priority subset. Full live AI generation remains gated only by model/API configuration.

已有证据包括：t1/sigmoid_scale_sum 手动 Pass@4 全部正确，最佳候选达到约 2.0279x 加速；同一任务通过 replay provider 复现生成链路，最佳候选约 1.998x；t2/add_rmsnorm_cast 和 t3/layernorm_gated 形成正向 T2/T3 normalization 案例；t1/fused_silu_and_mul、t1/softmax 和 t2/add_rmsnorm_quant 形成性能或精确量化边界负例；GELU 调优形成了数值 稳定性、UB 压力、Triton-Ascend backend 诊断等可复用经验。

本项目的核心贡献是一个模型无关、后端可扩展、实验可追踪的算子生成系统，而不是单个算子的临时优化。项目书完整版在基础版基础上补充了当前实现进度、技术设计、复现实验、提交包装方案和下一步计划。

一、团队介绍

本团队围绕 AI/Agent 驱动的高性能算子自动生成开展工作，目标后端为昇腾 910 NPU。团队成员关注编译系统、AI 芯片架构、IR/Pass 工程、算子调优、自动化验证、大模型辅助软件工程和工程化实验复现，具备开发本项目所需的系统实现能力。

团队前期能力与本项目匹配点包括：

- 理解昇腾 NPU 算子运行链路，包括 CANN、ACL Runtime、算子编译、运行、profiling 和日志分析。
- 关注 Ascend C、Triton、TileLang、MLIR 等算子语言与编译基础设施，理解 tile、访存、并行、同步、流水线和硬件资源约束。
- 具备代码生成 Agent、RAG、Prompt/Skill 设计、自动化测试、错误归因和修复流程的开发经验。
- 希望通过比赛沉淀可复用的 NPU 算子生成技能库和硬件反馈闭环，而不是只完成若干单点算子。

二、项目背景与研究现状

大模型结构、AI 加速硬件和编译器栈持续迭代，使高质量算子的需求不断增长。传统算子开发依赖专家手工设计 kernel、tile 策略、数据搬运、边界处理、编译调试和性能分析，开发成本高，并且难以快速覆盖新模型、新 shape 和新硬件后端。

近年来，LLM 代码生成和 Agent 工作流为“自动生成高性能算子”提供了新的路径。KernelBench 将 LLM 写 CUDA/DSL kernel 作为系统评测任务，强调生成代码必须同时正确和高性能。AKG Kernel Agent 则进一步探索多 Agent、跨 DSL、跨硬件后端的 kernel 生成、迁移和调优。

在昇腾 NPU 场景中，直接让 LLM 生成底层 Ascend C 或复杂 kernel 更困难，原因包括：

- Ascend C API 与 host-side tiling 约束强。
- 公开高质量样例比 CUDA 生态少。
- NPU 的 GM/UB 数据搬运、流水线和 backend API 语义与 GPU 不完全一致。
- 性能依赖真实硬件 profiling，CPU 模拟无法替代。
- Benchmark 结果需要正确性与性能共同验证，不能只依赖静态代码质量。

因此，本项目采用结构化中间层和硬件反馈闭环：先将 Benchmark 任务转成 OpSpec 和 NPU-aware Sketch，再由 provider 和 Code Agent 生成候选代码，通过官方 runner 在真实 NPU 上验证和记录，最后把成功和失败经验写回 Skill Library。

三、项目定位与核心目标

项目名称为 **SketchSkill-AGK**：面向昇腾 NPU 的技能驱动算子自动生成与硬件反馈优化系统。

最终产品不是单个优化 kernel，也不是一组手写提交文件，而是一个模型无关的 Agent 系统原型，以及可复现的 Benchmark 证据。系统应能在同一套 OpSpec、Sketch、Skill、Prompt、候选元数据、Benchmark runner 和结果导入流程下比较不同 provider，例如确定性 replay 以及后续本地/open code 模型。

核心目标如下：

- 算子生成目标：从 Benchmark 描述、参考实现、shape/dtype 约束生成可执行 NPU ModelNew 候选，当前主路径为 Triton-Ascend。
- 正确性目标：在昇腾 NPU 上编译运行候选，与标准实现对比，记录 Pass@1、Pass@4、误差、失败 shape 和错误类型。
- 性能目标：只对正确性通过的候选测量 latency、speedup 和 score，引入 tile、并行映射、访存和流水线搜索。
- AI 技术目标：实现 Spec、Sketch、Skill Retriever、Code、Verify、Repair、Profiler/Search 和 Skill Writer 的可组合工作流，其中 LLM 是可替换 provider。
- 资产沉淀目标：形成 SKILL.md、Prompt 模板、错误修复规则、调优规则、bad-to-good 轨迹和实验复现脚本。
- 提交目标：形成 GitLink PR 包、项目书完整版、技术设计文档、Benchmark 证据和复现说明。

四、总体技术路线

项目路线为：

```
AKG Bench Lite task / reference Model
-> Benchmark Parser
-> OpSpec
-> NPU-aware Operator Sketch
```

```
-> Skill Retriever
-> Provider / Code Agent
-> ModelNew candidate
-> submission materialization
-> backend probe
-> official correctness/performance benchmark
-> Pass@N report
-> failure classification / repair / tuning
-> Skill Library write-back
```

系统坚持两条原则：

1. LLM 负责需要推理和生成的环节，例如 Sketch 规划、代码生成、修复建议和技能总结。
2. Benchmark 扫描、AST 解析、OpSpec schema、正确性比较、score 导入、Pass@N 汇总和最佳候选选择尽量确定化、可测试化，避免把评测结果建立在模型自由输出上。

当前主路径选择 **AKG Bench Lite + Triton-Ascend + 官方 `run\_bench.py` runner**。增强路径保留 TileLang-Ascend、Ascend C、CCE/TBE 知识、MLIR/AscendNPU IR 对齐和 CUDA/Triton 经验迁移。

五、当前实现进展

截至 2026 年 7 月 8 日，项目已从基础方案推进到可运行原型，关键实现如下。

5.1 Benchmark 与环境复现

- 官方 AKG 仓库以 Git submodule 固定在 `third_party/akg`。
- Benchmark 源为 `akg_agents/benchmark/akg_kernels_bench_lite`。
- 固定分支与提交：`br_agents / 47aa428fcdc8c68f78d331dc578bc6c74fb9d91d`。
- 提供 `scripts/setup_benchmark_submodule.sh` 初始化 Benchmark。
- 提供 `scripts/bootstrap_ascend_env.sh` 在昇腾 worker 上重建 `venv`、`CANN` 环境、`Triton-Ascend backend` 和诊断流程。
- 已在真实昇腾设备上运行官方 Benchmark，记录硬件和软件版本。

当前已记录的主要硬件环境：

项目	值
NPU	Ascend910B2C
CANN	8.5.1
Python	3.11.14

torch	2.9.0+cpu
torch_npu	2.9.0rc1
Triton-Ascend	3.2.0

5.2 Benchmark Registry、OpSpec 与 Sketch

项目已扫描官方 13 个 `akg_kernels_bench_lite` case，并形成 registry：

Tier	数量
T1	6
T2	4
T3	3

当前自动 OpSpec/Sketch 覆盖已达到 Lite benchmark 13/13：

- T1: `gelu`、`fused_silu_and_mul`、`matmul_basic`、
`matmul_biasadd`、`sigmoid_scale_sum`、`softmax`。
- T2: `add_rmsnorm_cast`、`add_rmsnorm_quant`、
`moe_topk_softmax`、`rope`。
- T3: `causal_conv1d`、`decode_mla`、`layernorm_gated`。

`benchmarks/raw/akg_kernels_bench_lite_registry.yaml` 记录 13 个 case，`benchmarks/parsed/` 下有对应 13 个 OpSpec YAML。`scripts/validate_opspecs.py` 会检查必需字段、`tensor spec`、`validation/performance` 字段、非泛化 Sketch、`tuple-output contract` 和 `matmul/MoE` 等类别约束；当前结果为 13/13 通过。

OpSpec 记录内容包括：

- `task id`、`tier`、算子类别、源码路径；
- 输入输出 `tensor name`、`shape`、`dtype`、`layout`；
- 算子语义、`broadcast`、`reduction axes`、`normalization axes`、`layout transform`；

- rtol/atol、shape/dtype cases;
- 官方 Benchmark 性能参数;
- 对应 NPU-aware Sketch;
- 提交文件布局与 ModelNew entrypoint。

Sketch 字段包括:

- operator_category
- compute_pattern
- parallel_axes
- tile_plan
- memory_plan
- pipeline_plan
- boundary_mask
- accumulation_dtype
- backend_target
- performance_knobs
- known_risks

5.3 Candidate、Submission 与 Result Pipeline

项目已实现:

- 通用提交布局生成: scripts/create_submission.py 和 kernel_forge/submission.py。
- Benchmark JSON 导入: scripts/import_benchmark_result.py。
- Pass@N 汇总: scripts/summarize_passn.py。
- backend probe: GELU、sigmoid_scale_sum、fused_silu_and_mul 均有独立 probe 脚本。
- 实验记录 schema: experiments/runs/*.yaml。
- 汇总报告: experiments/reports/*.yaml 和 experiments/reports/gelu_tuning_summary.md。

5.4 Provider 与 Prompt Workflow

项目已实现第一个 provider 边界:

- ProviderRequest: 包含 case id、candidate index、Pass@N、backend、prompt version、OpSpec、Sketch、retrieved skills 和元数据。

- `ProviderResponse`: 包含候选源码文本、`provider`、`model` 和 `provider metadata`。
- `replay provider`: 确定性回放已验证候选，用于 CI/regression 和 pipeline 验证。
- `openai provider`: 通过 OpenAI Responses API 接入 live LLM，凭证和模型名来自环境变量，测试通过 fake HTTP poster，不依赖真实网络和密钥。

Prompt 模板已存在：

- `prompts/code_agent.v1.md`
- `prompts/repair_agent.v1.md`
- `prompts/skill_writer.v1.md`

生成命令示例：

```
python scripts/generate_candidate.py \
  --opspec benchmarks/parsed/t1_sigmoid_scale_sum.yaml \
  --provider replay \
  --backend triton_ascend \
  --pass-n 4 \
  --run-id replay-sigmoid-scale-sum-pass4 \
  --output-root outputs/generated
```

5.5 Skill Library

技能库已按算子模式和工程任务组织：

```
skills/
  elementwise/SKILL.md
  broadcast/SKILL.md
  reduction/SKILL.md
  transpose_layout/SKILL.md
  normalization/SKILL.md
  matmul_like/SKILL.md
  ascend_debug/SKILL.md
  ascend_performance/SKILL.md
  cuda_to_ascend_migration/SKILL.md
  benchmark_evaluation/SKILL.md
```

已沉淀的实验经验包括：

- GELU 数值稳定形式：避免 $0.5 * x * (1 + \tanh(u))$ 在负尾部取消误差，使用稳定 sigmoid 等价形式。
- Triton-Ascend backend 诊断：区分真实 Triton path 与 PyTorch fallback path。
- UB 压力边界：GELU block size 32768 出现 UB overflow，24576 可编译。
- Rowwise reduction 正向案例：sigmoid_scale_sum 在整行 8192 归约轴可放入一个 tile 时，一行一个 program、完整归约 tile 速度最好。

- Fused SwiGLU 负向案例：fused_silu_and_mul 的 Triton flattened-output 变体可正确但显著慢于框架 intrinsic，应记录为性能负例。

六、关键技术设计

完整技术设计见 docs/technical_design.md。本节概述项目书中的关键设计。

6.1 OpSpec：稳定的算子语义接口

OpSpec 是 Benchmark task 到生成系统的确定性接口。它把原始 Python reference、输入构造和 runner 约束转换为结构化 YAML。这样做的目标是让后续 Agent 不直接解析复杂源文件，而是面对稳定、可测试、可 diff 的语义记录。

示例字段：

```
id: t1/sigmoid_scale_sum
category: reduction
inputs:
  - name: x
    shape: [1000, 8192]
    dtype: float32
  - name: bias
    shape: [8192]
    dtype: float32
outputs:
  - name: output
    shape: [1000, 1]
semantics:
  formula: sum(sigmoid(x * 2.0 + bias), dim=-1, keepdim=True)
validation:
  rtol: 0.01
  atol: 0.01
```

6.2 NPU-aware Operator Sketch

Sketch 是项目核心中间表示，用于连接算子语义和后端代码。它不是完整 DSL，而是 JSON/YAML 风格的结构化计划，便于 LLM 生成、检查、修复和检索。

不同算子类别关注不同字段：

类别	主要关注
elementwise	连续访存、flattened output elements、尾部 mask、数学表达式 backend lowering
fused elementwise	多输入索引、split/layout、输出连续写、重复索引开销

reduction	reduction axis、accumulation dtype、row mapping、partial reduction、keepdim
softmax/normalization	max-subtraction、sum reduction、数值稳定性、归一化轴
transpose/layout	读写连续性、tile 重排、bank conflict 风险
matmul-like	M/N/K tile、数据复用、double buffer、copy/compute overlap

6.3 Skill Retrieval 与 Prompt 约束

Skill Retriever 根据 OpSpec category、Sketch category、broadcast 和 layout transform 选择技能文件。例如 sigmoid_scale_sum 会检索：

- skills/reduction/SKILL.md
- skills/broadcast/SKILL.md
- skills/ascend_debug/SKILL.md
- skills/ascend_performance/SKILL.md
- skills/benchmark_evaluation/SKILL.md

Prompt 会包含 OpSpec YAML、Sketch YAML、候选序号、Pass@N、backend 和技能摘要，要求 provider 返回单个 Python source file，并定义 ModelNew。

6.4 Provider 边界

Provider 边界把模型调用从工程流程中隔离出来：

- replay provider 用于确定性验证 pipeline。
- openai provider 用于 live LLM 生成，未来可加入本地/open code model provider。
- provider 输出必须写入候选文件、prompt 文件、metadata JSON 和 experiment YAML。
- experiment 记录必须包含 provider、model、prompt version、retrieved skills、candidate index 和 candidate path。

6.5 Correctness、Pass@N 与 Performance

正确性是硬门槛。Benchmark 默认：

- rtol = 1e-2
- atol = 1e-2

- 同时记录 `max_abs_diff` 和 `max_rel_diff`

Pass@N 定义：

- Pass@1：第一个候选通过正确性。
- Pass@4：四个候选中至少一个通过正确性。

性能只在正确性通过后解释，记录：

- baseline latency
- solution latency
- speedup
- weighted score
- backend probe 结果
- 是否为真实 Triton-Ascend path

七、当前 Benchmark 证据

7.1 GELU 调优案例

GELU 是第一个端到端调试案例。初始候选曾出现 PyTorch fallback、Triton backend 缺失、相对误差失败、UB overflow、不同数学 lowering 性能差异等问题。

最佳已记录真实 Triton-Ascend 候选为 `gelu_triton_v13`：

Candidate	正确性	Speedup	Score	说明
<code>gelu_triton_v7</code>	pass	0.0728x	4.37	第一个纯 Triton 正确性通过
<code>gelu_triton_v10</code>	pass	0.5635x	33.81	block size 16384
<code>gelu_triton_v11</code>	compile fail	-	-	UB overflow
<code>gelu_triton_v13</code>	pass	0.6059x	36.35	当前最佳 GELU Triton 候选
<code>gelu_triton_v18</code>	pass	0.5764x	34.58	exp2 lowering 未超过 v13

结论：GELU 已形成有价值的数值和 backend 调试案例，但当前 Triton-Ascend 版本仍慢于 PyTorch baseline，不适合作为下一阶段单算子调优主目标。

7.2 t1/sigmoid_scale_sum 手动 Pass@4

任务语义：

```
sum(sigmoid(x * 2.0 + bias), dim=-1, keepdim=True)
shape: x [1000, 8192], bias [8192], output [1000, 1]
```

官方 Benchmark 结果：

Candidate	Backend 策略	正确性	Speedup	Score
sigmoid_scale_sum_v1	torch reference	pass	1.0006x	60.01
sigmoid_scale_sum_v2	Triton one row/program, 8192 tile	pass	2.0279x	70.28
sigmoid_scale_sum_v3	Triton 2 x 4096 chunks	pass	1.9367x	69.37
sigmoid_scale_sum_v4	Triton 4 x 2048 chunks	pass	1.5785x	65.79

结论：这是项目第一个非 GELU 正向性能案例，说明 rowwise reduction 在 reduction axis 可完整放入一个 tile 时，Triton-Ascend 自定义 kernel 可以超过框架 baseline。

7.3 t1/sigmoid_scale_sum Replay Provider Pass@4

Replay provider 使用同一 provider request、prompt、skill retrieval、candidate metadata 和 submission materialization 流程，返回已知候选模板。其作用不是证明模型质量，而是证明生成 pipeline 和实验追踪链路可复现。

Candidate	正确性	Speedup	Score
sigmoid_scale_sum_replay_v1	pass	1.0034x	60.03
sigmoid_scale_sum_replay_v2	pass	1.9980x	69.98
sigmoid_scale_sum_replay_v3	pass	1.8731x	68.73
sigmoid_scale_sum_replay_v4	pass	1.5451x	65.45

结论：replay provider 已能支撑确定性 CI/regression，并证明 provider 边界、prompt、candidate、submission 和 experiment record 链路可用。

7.4 t1/fused_silu_and_mul Pass@4

任务语义：

```
silu(combined[... , :H]) * combined[... , H:]
shape: combined [4096, 8192], output [4096, 4096]
```

官方 Benchmark 结果：

Candidate	策略	正确性	Speedup	Score
fused_silu_and_mul_v1	torch / intrinsic reference	pass	1.0027x	60.03
fused_silu_and_mul_v2	Triton flattened tile	pass	0.0033x	0.20
fused_silu_and_mul_v3	Triton reduced UB pressure	pass	0.0033x	0.20
fused_silu_and_mul_v4	Triton chunked variant	pass	0.0033x	0.20

结论：该任务正确性可达成，但当前 Triton flattened-output 思路性能很差。项目把它作为负向性能案例写入 Skill Library，用于提示后续 Code Agent 不应盲目替代高效 NPU intrinsic。

八、创新点

8.1 NPU-aware Operator Sketch

项目将算子生成拆分为语义规划和代码实现。Sketch 显式表达算子类别、并行轴、tile plan、memory plan、pipeline plan、boundary mask、accumulation dtype 和 performance knobs，使 LLM 不必直接面对复杂底层 API，从而降低 hallucination、shape 错误和 backend 语义错误。

8.2 实验驱动的 Skill Library

Skill Library 不是静态提示词集合，而是由真实 Benchmark 实验驱动更新的工程 playbook。它包含适用条件、shape/dtype 约束、Sketch 模板、tile 策略、访存策略、常见错误、profiling 解释和 bad-to-good 轨迹。

8.3 正确性与性能双闭环

系统将正确性和性能分成两个闭环。第一层通过编译日志、运行错误和数值误差修复候选；第二层只对正确性通过的候选做硬件 profiling 和性能搜索。这样避免为了性能牺牲正确性，也避免把 fallback 结果误判为自定义 kernel 成功。

8.4 Provider 可替换

项目明确把 LLM 作为 provider，而不是系统本身。replay 用于确定性回归，指定模型用于 live LLM，未来可接入本地模型。所有 provider 共用同一 OpSpec、Sketch、Skill、Prompt、Benchmark 和结果导入流程，使模型比较更公平。

8.5 正负样本都写回

项目不仅记录成功加速案例，也记录性能负例。例如 fused_silu_and_mul 的 Triton 变体正确但慢，这类信息对后续生成同样重要，可以防止 Agent 重复走低价值方向。

九、AI 技术选择

项目采用 Agent/RAG/Prompt/Skill 的组合，不把大规模微调作为初赛主路线。原因是比赛周期短、NPU 领域高质量训练数据有限，且真实硬件验证比离线训练更关键。

当前 AI 技术方案：

- **Agent workflow:** Spec、Sketch、Skill Retriever、Code、Verify、Repair、Profiler/Search、Skill Writer。
- **RAG/Skill retrieval:** 从本地 skills/、实验记录、Prompt 模板和后续外部文档检索上下文。
- **Prompt engineering:** 用 OpSpec YAML、Sketch YAML 和技能摘要约束 Code Agent 输出。
- **Provider abstraction:** 同一接口支持 replay、OpenAI、未来本地/open code model。
- **硬件反馈搜索:** 基于真实 NPU 正确性和性能结果迭代，不依赖模型主观判断。

十、代码与复现说明

10.1 初始化 Benchmark

```
bash scripts/setup_benchmark_submodule.sh
```

10.2 扫描 Benchmark Registry

```
python scripts/scan_benchmark_cases.py \
  --output benchmarks/raw/akg_kernels_bench_lite_registry.yaml \
  --repo-root .
```

10.3 生成 OpSpec 和 Sketch

```
python scripts/extract_opspec_batch.py \
  --output-dir benchmarks/parsed \
  --repo-root .
```

10.4 通过 Provider 生成候选

```
python scripts/generate_candidate.py \
  --opspec benchmarks/parsed/t1_sigmoid_scale_sum.yaml \
  --provider replay \
  --backend triton_ascend \
  --pass-n 4 \
  --run-id replay-sigmoid-scale-sum-pass4 \
  --output-root outputs/generated
```

10.5 生成官方提交布局

```
python scripts/create_submission.py \
  --team sigmoid_scale_sum_v2 \
  --candidate sigmoid_scale_sum_v2 \
  --case t1/sigmoid_scale_sum=kernel_forge/candidates/sigmoid_scale_sum_v2.py
```

10.6 运行官方 Benchmark

```
python third_party/akg/akg_agents/benchmark/akg_kernels_bench_lite/tools/run_bench.py \
  outputs/submissions/sigmoid_scale_sum_pass4 \
  --bench-dir third_party/akg/akg_agents/benchmark/akg_kernels_bench_lite \
  --output outputs/results/sigmoid_scale_sum_pass4 \
  --warmup 10 \
  --iterations 100 \
  --num-trials 3
```

10.7 汇总 Pass@N

```
python scripts/summarize_passn.py \
  --results-dir outputs/results/sigmoid_scale_sum_pass4 \
  --case t1/sigmoid_scale_sum \
  --candidate sigmoid_scale_sum_v1 \
  --candidate sigmoid_scale_sum_v2 \
  --candidate sigmoid_scale_sum_v3 \
  --candidate sigmoid_scale_sum_v4 \
  --output experiments/reports/2026-07-07-sigmoid-scale-sum-pass4.yaml
```

10.8 本地测试

```
python -m pytest -q tests/test_agent_generation_workflow.py tests/
test_fused_silu_and_mul_pass4.py
```

完整测试在初始化 AKG submodule 后运行：

```
python -m pytest -q
```

十一、实施计划

阶段	当前状态	下一步
环境与 Benchmark 复现	已完成基本闭环	继续记录不同 Ascend worker 差异
OpSpec 与 Sketch	Lite benchmark 13/13 已支持并通过 validator	扩展动态 shape 和更大 Benchmark 覆盖
Candidate 与 Pass@N	priority subset 手动和 replay Pass@4 已跑通	接入 live provider 生成并验证
Skill Library	matmul、top-k softmax、normalization、reduction、layout 等正负案例已写回	增加 prompt-to-skill 自动总结
Repair Agent	规则和 prompt 初版存在	实现自动错误分类和修复循环
Profiler/Search	手动 tuning 案例存在	增加 profile import 与搜索记录

十二、预期成果与评价指标

预期成果：

- SketchSkill-AGK 原型系统。
- Benchmark registry、13/13 Lite OpSpec、Sketch、Prompt、Skill 和候选生成 pipeline。
- 官方 Benchmark 正确性和性能报告，当前覆盖 priority subset。
- Pass@1/Pass@4 汇总。
- 至少一个正向性能案例和一个负向性能案例。
- 可复现的远程 Ascend 实验记录。
- GitLink PR、项目书完整版和技术设计文档。

评价指标：

- 编译成功率。
- runtime 成功率。
- correctness pass rate。

- Pass@1 / Pass@4。
- max_abs_diff / max_rel_diff。
- baseline latency / solution latency。
- speedup / weighted score。
- provider、model、prompt version、retrieved skills 是否可追踪。
- 失败是否能归类并写回技能库。

十三、风险分析与应对

风险	影响	应对
LLM 直接生成代码不稳定	编译失败、API 幻觉	使用 OpSpec、Sketch、Skill 和 Prompt 约束；保留 replay regression provider
正确性不足	Pass@N 不达预期	多候选生成、backend probe、错误分类、Repair Agent
性能不及 baseline	score 低	正确性优先；只对正确候选做 profiling；记录负例防止重复
Triton-Ascend API 差异	跨 worker 不稳定	记录 CANN、torch_npu、triton、backend 版本；提供 bootstrap 脚本
NPU 资源有限	搜索轮次不足	保持搜索空间小，优先 T1 子集和代表性算子
更大 Benchmark 或动态 shape 未覆盖	后续泛化证据不足	当前 Lite 13/13 已完成；后续扩展更复杂 shape 和输入构造

十四、参考文献与相关开源项目

[1] MindSpore AGK. Auto Kernel Generator (AGK) README.

[2] MindSpore AGK Agents. AGK Agents README_CN.

[3] Jinye Du et al. AGK Kernel Agent: A Multi-Agent Framework for Cross-Platform Kernel Synthesis. arXiv:2512.23424, 2025.

[4] Scaling Intelligence, Stanford. KernelBench: Can LLMs Write GPU Kernels? 2024/2025.

- [5] KernelBench authors. KernelBench: Can LLMs Write Efficient GPU Kernels? arXiv:2502.10517, 2025.
- [6] Zhongzhen Wen et al. AscendCraft: Automatic Ascend NPU Kernel Generation via DSL-Guided Transcompilation. arXiv:2601.22760, 2026.
- [7] AscendOptimizer authors. AscendOptimizer: Episodic Agent for Ascend NPU Operator Optimization. arXiv:2603.23566, 2026.
- [8] Tile-AI. TileLang and TileLang-Ascend documentation and repositories.
- [9] Triton Project. Triton language and compiler documentation.
- [10] LLVM MLIR Project. Linalg Dialect documentation.
- [11] Huawei / MindSpore. Ascend C Custom Operator Development and CANN documentation.
- [12] KrxGu. kernel-skills: skill library for coding agents to write, optimize and debug compute kernels.
- [13] Kevin authors / Cognition. Kevin: Multi-Turn RL for Generating CUDA Kernels. arXiv:2507.11948, 2025.
- [14] CUDA-L1 authors. Improving CUDA Optimization via Contrastive Reinforcement Learning. 2025.

附录 A：技术设计文档

SketchSkill-AGK 技术设计文档

本文档是初赛提交包中的技术设计说明，面向评审解释 SketchSkill-AGK 的系统边界、模块设计、数据结构、Agent/provider 工作流、验证流程和当前实现状态。

1. 设计目标

SketchSkill-AGK 的目标是在昇腾 910 NPU Benchmark 上构建一个可复用的 AI/Agent 算子生成系统。系统输入是官方 Benchmark 任务和参考实现，输出是可提交到 AGK Bench Lite 的 ModelNew 实现、正确性与性能结果、Pass@N 报告和可复用技能。

系统不把 LLM 作为不可替代的中心，而把 LLM 作为 provider。确定性模块负责 Benchmark 扫描、AST 解析、OpSpec schema、Sketch 模板、提交布局、结果导入、Pass@N 统计和实验记录；provider 负责候选代码生成、修复建议和技能总结等需要生成能力的环节。

2. 总体架构

```
AGK Bench Lite task
-> Benchmark Parser
-> OpSpec
-> NPU-aware Operator Sketch
-> Skill Retriever
-> Prompt Renderer
-> Provider / Code Agent
-> Candidate Source
-> Submission Layout
-> Backend Probe
-> Official Benchmark Runner
-> Result Importer
-> Pass@N Reporter
-> Repair / Search / Skill Writer
```

当前实现位于：

模块	路径
Benchmark parser / OpSpec / Sketch	kernel_forge/benchmark/
Provider / Prompt / Skill retrieval workflow	kernel_forge/agents/
Submission layout helper	kernel_forge/submission.py
Result import and Pass@N	kernel_forge/experiments/

Candidate examples	kernel_forge/candidates/
CLI scripts	scripts/
Prompt templates	prompts/
Skill Library	skills/
Experiment records	experiments/
Validation gates	kernel_forge/benchmark/validation.py, scripts/ validate_opspecs.py

3. Benchmark 与 OpSpec 设计

官方 Benchmark 固定为：

```
repo: https://atomgit.com/mindspore/akg.git
branch: br_agents
commit: 47aa428fcdc8c68f78d331dc578bc6c74fb9d91d
path: akg_agents/benchmark/akg_kernels_bench_lite
```

项目通过 submodule 和 sparse checkout 管理 Benchmark，避免直接 vendoring 第三方代码。

OpSpec 是 Benchmark task 到 Agent pipeline 的稳定接口。字段包括：

字段	作用
id, name, tier, category	标识任务和算子类别
source_path	官方 reference 源文件
inputs, outputs	tensor 名称、shape、dtype、layout
semantics	公式、broadcast、reduction axes、layout transform
validation	rtol、atol、shape/dtype cases
performance	warmup、repeats、baseline source
sketch	NPU-aware Operator Sketch
submission	需要提交的文件和 entrypoint

当前 OpSpec/Sketch 覆盖：

```
Lite benchmark OpSpec coverage: 13/13.
```

覆盖包括 T1 的 GELU、fused SiLU、matmul、softmax/reduction，T2 的 RMSNorm、MoE top-k softmax、RoPE，以及 T3 的 causal conv1d、decode MLA 和 layernorm gated。scripts/validate_opspecs.py 对 13 个 parsed YAML 执行 结构验证，包括必需字段、tensor spec、validation/performance 字段、非泛化 Sketch、tuple-output contract 和 matmul/MoE 等类别约束。

4. NPU-aware Operator Sketch

Sketch 是本系统的核心中间层。它不是完整 DSL，而是结构化计划，用于降低 LLM 直接生成底层 NPU kernel 的难度。

统一字段：

```
operator_category: reduction
compute_pattern: sigmoid_scale_sum_row_reduction
parallel_axes:
  - outer_rows
tile_plan:
  strategy: rowwise_last_dim_tiling
  shape: [1000, 8192]
  tunable: true
memory_plan:
  input: row_contiguous_global_read
  output: contiguous_global_write
  intermediate: local_vector_reduction
pipeline_plan:
  stages:
    - load_x_and_bias
    - compute_sigmoid
    - row_sum
    - store
boundary_mask:
  required: true
accumulation_dtype: float32
backend_target: triton_ascend
performance_knobs:
  row_tile: null
  reduction_tile: null
known_risks:
  - broadcast_bias_correctness
  - reduction_accuracy
```

Sketch 的作用：

- 将算子语义转为后端可实现的并行和访存计划。
- 为 Prompt 提供稳定上下文。
- 为 Repair Agent 提供可定位的修复对象。
- 为 Search Agent 提供可解释的调优旋钮。
- 为 Skill Library 写回提供结构化挂载点。

5. Skill Library 与检索策略

Skill Library 是项目的可复用知识资产。每个 SKILL.md 包含适用边界、shape/dtype 约束、Sketch 建议、后端生成注意事项、常见错误、profiling 解释和 bad-to-good 案例。

检索策略当前为规则化选择：

- reduction 算子检索 skills/reduction/SKILL.md。
- elementwise / fused elementwise 检索 skills/elementwise/SKILL.md。
- normalization 算子检索 skills/normalization/SKILL.md。
- matmul-like 算子检索 skills/matmul_like/SKILL.md。
- transpose/layout 算子检索 skills/transpose_layout/SKILL.md。
- 存在 broadcast 时检索 skills/broadcast/SKILL.md。
- 存在 layout transform 时检索 skills/transpose_layout/SKILL.md。
- 所有生成任务检索 skills/ascend_debug/SKILL.md、skills/ascend_performance/SKILL.md 和 skills/benchmark_evaluation/SKILL.md。

该策略简单但可测试，后续可替换为 embedding/RAG 检索。

6. Provider 和 Prompt 设计

Provider 接口定义在 kernel_forge/agents/provider.py。

6.1 ProviderRequest

ProviderRequest 包含：

- case id
- candidate index
- Pass@N
- backend target
- prompt version
- rendered prompt
- OpSpec
- Sketch
- retrieved skill paths

- run metadata

6.2 ProviderResponse

ProviderResponse 包含：

- generated source text
- provider name
- model name
- response metadata

6.3 已实现 provider

Provider	作用
replay	确定性回放已验证候选，证明 pipeline、metadata、submission 和 experiment flow
openai	OpenAI Responses API live provider，凭证和模型通过环境变量配置

openai provider 的测试不访问网络，而是注入 fake HTTP poster，验证请求构造、响应解析、code fence 清理和缺失配置错误。

7. Candidate Generation Workflow

scripts/generate_candidate.py 调用 generate_passn_candidates，流程如下：

10. 读取 OpSpec。
11. 根据 OpSpec 检索技能。
12. 渲染 prompts/code_agent.v1.md。
13. 构造 ProviderRequest。
14. 调用 provider。
15. 写入候选源码。
16. 写入 prompt 文件和 metadata JSON。
17. 生成 official submission layout。
18. 写入 experiment YAML。

生成 artifacts 默认位于 outputs/generated/，不提交到 Git；提交只保留 provider 代码、Prompt、Skill、测试和简洁实验记录。

8. 正确性验证设计

官方 runner 对每个 candidate 执行：

- reference Model 与 candidate ModelNew 输出对比；
- output 类型、数量、shape 检查；
- max_abs_diff 和 max_rel_diff；
- 默认 $rtol = 1e-2$, $atol = 1e-2$ ；
- 正确性通过后才测量性能。

项目额外要求：

- backend probe 必须记录 `_last_backend`，区分真实 Triton path 和 fallback path；
- experiment record 必须保存 correctness、performance、pass_n 和 artifact path；
- Pass@N 汇总必须保留所有候选，而不是只保留最佳候选。

9. 性能验证与搜索设计

性能指标：

- baseline median latency
- solution median latency
- speedup
- weighted score
- warmup / iterations / num_trials
- backend path

搜索只在正确性通过后进行。初始搜索空间包括：

- tile size
- row/reduction tile
- sequential chunks
- parallel axis
- vector width
- unroll factor
- double buffering
- boundary strategy

当前项目已有手动搜索案例：

- GELU block size 从 1024 到 16384 持续改善，32768 触发 UB overflow，16384 x 2 chunks 为当前最佳。
- sigmoid_scale_sum 一行一个 program、完整 8192 reduction tile 最优，拆成多个 chunk 反而变慢。
- fused_silu_and_mul 的 flattened-output Triton 变体正确但极慢，应优先考虑框架 intrinsic 或不同后端策略。
- add_rmsnorm_cast 的 4096-wide rowwise RMSNorm Triton seed 形成正向 T2 normalization 案例。
- layernorm_gated 表明 fp16 中间舍入语义会影响正确性，row grouping 可在正确性固定后提升性能。

10. Repair Agent 设计

Repair Agent 的目标是根据错误类别路由修复：

错误类别	路由
syntax/API/type	Code Agent
shape/broadcast/layout	Sketch Agent + Code Agent
tile/memory/boundary	Sketch Agent
numerical	Sketch Agent 检查 dtype、mask、reduction、数学形式
environment/backend	环境 owner，不计为模型失败
performance regression	Profiler/Search Agent

当前已有 repair prompt 和技能规则，完整自动修复循环仍是后续工作。

11. Skill Writer 设计

Skill Writer 从实验记录中抽取可复用经验：

- 失败现象。
- root cause。

- 修复策略。
- 适用 shape/dtype/backend。
- before/after correctness 和 latency。
- candidate path、experiment path 和 report path。

当前写回以人工整理为主，已覆盖 GELU、sigmoid_scale_sum、fused_silu_and_mul、softmax、add_rmsnorm_cast、add_rmsnorm_quant、RoPE、layernorm_gated、matmul 和 MoE top-k softmax 的关键经验。后续将让 Skill Writer 自动生成候选写回内容，再由人审阅。

12. 当前实验证据

任务	Pass@1	Pass@4	最佳候选	Speedu	说明
t1/gelu	是	未按正式 Pass@4 统计	gelu_triton_v13	0.6059x	正确性正例，性能慢于 baseline
t1/ sigmoid_scale_sum manual	是	是，4/4	sigmoid_scale_sum_v2	2.0279x	第一个正向非 GELU 加速案例
t1/ sigmoid_scale_sum replay	是	是，4/4	sigmoid_scale_sum_replay_v2	1.9980x	provider pipeline 复现
t1/ fused_silu_and_mul	是	是，4/4	fused_silu_and_mul_v1	1.0027x	Triton 变体正确但性能负例
t1/softmax	是	是，4/4	softmax_v4	0.9225x	正确但慢于 baseline 的 rowwise softmax 经
t2/ add_rmsnorm_cast	是	是，4/4	add_rmsnorm_cast_v2	2.0135x	正向 T2 normalization 案例
t2/rope	是	是，4/4	rope_v1	1.0006x	NPU intrinsic 与 Triton parity 经验
t2/ add_rmsnorm_quant	是	是，1/4	add_rmsnorm_quant_v1	0.9981x	exact int8 量化边界负例
t3/layernorm_gated	是	是，4/4	layernorm_gated_v4	1.5137x	正向 T3 fp16 gated RMSNorm 案例

当前提交应采用的总括性表述为：

```
Lite benchmark OpSpec coverage: 13/13. Current executable candidates and
Pass@4 evidence cover a priority subset. Full live AI generation remains gated
only by model/API configuration.
```

13. 测试与质量控制

当前测试覆盖：

- Benchmark registry 和 OpSpec 提取。
- GELU OpSpec 和 YAML round-trip。
- generic submission layout。
- experiment result import。
- Pass@N summary。
- candidate Python 编译和安全 import 检查。
- provider workflow、OpenAI request/response parsing、replay generation。
- fused SiLU Pass@4 submission/report handling。
- full Lite OpSpec/Sketch validation through `scripts/validate_opspecs.py`。
- prompt context snapshots, package hygiene, replay guard, and result comparison preservation for multi-output cases。

本地建议运行：

```
python scripts/validate_opspecs.py --json
python -m pytest -q tests/test_pre_ai_infrastructure.py
```

初始化 submodule 后运行完整测试：

```
bash scripts/setup_benchmark_submodule.sh
python -m pytest -q
```

附录 B：提交包与复现说明

SketchSkill-AGK 初赛 提交包说明

当前提交包重点展示：

- 面向评审的中文包根目录 README.md
- Benchmark registry 和 Lite 13/13 OpSpec/Sketch 解析。
- scripts/validate_opspecs.py 确定性 validator，以及 prompt/package/replay 等 pre-AI infrastructure 测试。
- replay provider 和 openai provider 边界。
- Prompt 模板和 Skill Library。
- 官方 Ascend Benchmark 实验记录。
- Pass@N 报告和正负性能案例。

必含文件

- README.md
- docs/project_book_full_zh.md
- docs/technical_design.md
- docs/competition_alignment.md
- docs/architecture.md
- docs/project_workflow.md
- docs/benchmark_spec.md
- docs/dev_guide.md
- kernel_forge/
- scripts/
- prompts/
- skills/
- benchmarks/parsed/
- benchmarks/raw/akg_kernels_bench_lite_registry.yaml
- experiments/runs/*.yaml
- experiments/reports/*.yaml

初始化 Benchmark：

```
bash scripts/setup_benchmark_submodule.sh
```

扫描 Benchmark:

```
python scripts/scan_benchmark_cases.py \
  --output benchmarks/raw/akg_kernels_bench_lite_registry.yaml \
  --repo-root .
```

生成 OpSpec:

```
python scripts/extract_opspec_batch.py \
  --output-dir benchmarks/parsed \
  --repo-root .
```

运行 provider 生成 smoke:

```
python scripts/generate_candidate.py \
  --opspec benchmarks/parsed/t1_sigmoid_scale_sum.yaml \
  --provider replay \
  --backend triton_ascend \
  --pass-n 1 \
  --run-id replay-provider-smoke \
  --output-root /tmp/kf-generated-smoke
```

运行重点单元测试:

```
python -m pytest -q tests/test_agent_generation_workflow.py tests/
test_fused_silu_and_mul_pass4.py
```

在昇腾 worker 上运行官方 Benchmark 时使用:

```
python third_party/akg/akg_agents/benchmark/akg_kernels_bench_lite/tools/run_bench.py \
  outputs/submissions/sigmoid_scale_sum_pass4 \
  --bench-dir third_party/akg/akg_agents/benchmark/akg_kernels_bench_lite \
  --output outputs/results/sigmoid_scale_sum_pass4 \
  --warmup 10 \
  --iterations 100 \
  --num-trials 3
```